Checkpoint — Core Extraction & Project Restructure

Yassine — Amen Bank DCSI Internship

July 3, 2026

1 Session Objective

Refactor the streaming pipeline to separate reusable core logic from Kafka transport, enabling a shared codebase between the streaming path and a future FastAPI orchestration/BFF service for the simulation mode REST endpoint.

2 Architectural Decision: BFF / Orchestration Layer

The supervisor requested FastAPI integration for Docker deployment, covering both health/metrics endpoints on each streaming service and a REST `/score` endpoint for simulation mode.

Three options were evaluated for the synchronous `/score` endpoint:

Option	Description	Verdict
Request-reply via Kafka	Push event to `raw-events`, wait for result on `decisions` with correlation ID	Rejected: latency, complexity
Bypass Kafka	Duplicate logic or call services directly	Rejected: tight coupling, logic duplication
BFF/Orchestration service	Imports core computation from each service, calls enrich → score → decide in-process, returns synchronous response	**Selected**

The BFF pattern requires that core computation logic be cleanly separated from Kafka transport — motivating the full refactor below.

3 Design Pattern: Core Extraction

Each service was split into two layers following a consistent pattern:

1. **Core module** — Reusable computation logic. Receives external dependencies (Redis, model objects) via **dependency injection**. Exposes a single public method (façade pattern). Private helpers handle sub-steps (fetch, compute, update). No Kafka dependency.
2. **Streaming wrapper** — Thin Kafka consumer/producer plumbing. Receives a core instance via DI. The `run()` poll loop calls the core's public method and produces the result. No Redis, no models, no feature computation.

The `__main__` block is the sole wiring point: it creates the Redis client, model objects, core instance, and streaming wrapper, then starts the loop.

4 Refactored Modules

Core Module	DI Dependencies	Public Method	Internal Components
`src/enrichment/core.py` `EnrichmentCore`	Redis client	`enrich_event(event)`	Profile fetch, buffer management, 30 contrast features, LSTM sequence update
`src/scoring/core.py` `ScoringCore`	Redis client, ScoreFusion	`score_event(enriched)`	Feature extraction & scaling, sequence fetch & scaling, fusion scoring, conditional SHAP
`src/decision/core.py` `DecisionCore`	Redis client, DecisionService	`decide_event(scored)`	Profile fetch, DecisionInput construction, engine invocation, identifier attachment

Streaming Wrapper	Core Injected	Kafka Topics
`EnrichmentStreamingService`	EnrichmentCore	`raw-events` → `enriched-events`
`ScoringStreamingService`	ScoringCore	`enriched-events` → `scored-events`
`DecisionStreamingService`	DecisionCore	`scored-events` → `decisions`

5 Project Structure (Post-Refactor)

```
src/
  enrichment/
    __init__.py           -> exports EnrichmentCore
    core.py               -> feature computation + buffer management
  scoring/
    __init__.py           -> exports ScoringCore
    core.py               -> extract, scale, fuse, explain
    fusion.py             -> ScoreFusion (unchanged)
    ae_scorer.py          -> AEScorer (unchanged)
    lstm_scorer.py        -> LSTMScorer (unchanged)
  decision/
    __init__.py           -> exports DecisionCore
    core.py               -> profile fetch, input build, decide, output
    decision.py           -> DecisionService engine (unchanged)
    explainer.py          -> NLG templates (unchanged)
  streaming/
    enrichment_service.py -> thin Kafka wrapper
```

```
  scoring_service.py    -> thin Kafka wrapper
  decision_service.py   -> thin Kafka wrapper
  simulator.py          -> unchanged
api/                    -> FastAPI orchestration (next session)
batch/                  -> hydration, profiles, future Airflow
training/               -> model training
generator/              -> synthetic data
config.py               -> centralized config
```

Deleted: `src/enrichment.py` (stale batch version with pre-fix profile keys), `src/services/` (empty folder).

6 Key Design Decisions

1. **Dependency injection over internal construction.** Redis connections are injected so each caller (streaming, BFF, tests) controls its own connection lifecycle and pooling strategy.

2. **Façade pattern for workflow integrity.** Each core class exposes one public method. Sub-steps are private, preventing callers from reordering or skipping steps (e.g., computing features without updating buffers).

3. **Internal components stay internal.** `AEScorer` and `LSTMScorer` are created inside `ScoringCore` from the injected `ScoreFusion` — the caller doesn't need to know they exist. `__init__.py` re-exports only the core class.

4. **Core modules are delivery-agnostic.** `src/enrichment/` is not under `src/streaming/` because enrichment is feature computation, not a streaming concept. Both the streaming wrapper and the BFF import from the same core without semantically misleading import paths.

5. **Wiring happens at the edge.** The `__main__` block (or FastAPI `lifespan`) is the composition root — the only place that creates Redis clients, model objects, and assembles the dependency graph.

7 Next Session

1. Build FastAPI orchestration/BFF service (`src/api/`): POST `/score` imports EnrichmentCore → ScoringCore → DecisionCore

2. Add FastAPI health/metrics wrappers to streaming services (Kafka consumer as background task, `/health` endpoint)

3. Docker Compose: containerize all services

4. Streamlit dashboard (consumes decisions topic)